Guía del código del Generador de EMOM

JavaScript explicado línea por línea, versión simplificada y moderna

Este documento explica tu archivo **script.js** ya actualizado. Primero tienes la idea general, luego un pequeño diccionario y después el código troceado en tablas: código a la izquierda, significado a la derecha. Al final hay una comparación con la versión anterior y un mini test.

1. ¿Qué hace este programa?

Tienes un formulario donde eliges un ejercicio y escribes tu RM (el peso máximo que puedes levantar una vez). Al pulsar el botón, el programa dibuja una tabla de 14 minutos con las repeticiones y el peso que te toca en cada tramo, calculado como un porcentaje de tu RM.

El código hace tres cosas:

- **Validar:** si el RM es demasiado bajo o demasiado alto, muestra un mensaje de error propio.
- **Calcular:** para cada tramo multiplica tu RM por un porcentaje y lo redondea a discos de 2,5 kg.
- **Mostrar:** crea las filas de la tabla y hace visibles el resultado y el enlace.

Sobre los mensajes de error: en esta versión los textos son neutros y de ánimo. Puedes poner los que quieras, el código funciona igual con cualquier texto; si el proyecto lo va a ver alguien más (clase, portfolio), mejor que sean respetuosos.

2. El recorrido completo, paso a paso

Paso	Qué pasa	Qué parte del código lo hace
1	La persona escribe su RM en el campo.	El primer bloque (evento input)
2	Si el número es muy bajo o muy alto, aparece un error propio.	setCustomValidity
3	La persona pulsa el botón del formulario.	El evento submit
4	El código frena la recarga de la página y lee los datos.	e.preventDefault() y .value
5	Vacía la tabla y recorre las 7 fases calculando el peso de cada una.	replaceChildren() y FASES.forEach(...)
6	Crea una fila por fase y rellena sus celdas.	insertRow() e insertCell()
7	Hace visibles el resultado, la tabla y el enlace.	hidden = false

3. Mini diccionario antes de empezar

Estas palabras salen una y otra vez. Si las entiendes, el resto es fácil.

Palabra	Significado sencillo	Ejemplo
const	Crea una variable que no vas a reasignar. Es como una caja con etiqueta donde guardas un dato.	`const edad = 20;`
DOM	La página HTML convertida en objetos que JavaScript puede leer y modificar.	-
getElementById	Busca en la página el elemento que tiene ese id y te lo da.	`document.getElementById("rm")`
querySelector	Como el anterior, pero con selectores de CSS (#id, .clase, etiqueta).	`document.querySelector("#tablaEmom tbody")`
Evento	Algo que ocurre en la página: escribir, hacer clic, enviar un formulario.	`input, submit, click`
addEventListener	"Escucha" un evento y ejecuta una función cuando ocurre.	`boton.addEventListener("click", ...)`
Función flecha	Una función escrita con =>. Es un bloque de código que se ejecuta cuando lo llaman.	`() => { ... }`
Array	Una lista ordenada de cosas, entre corchetes.	`[10, 20, 30]`
Objeto	Un grupo de datos con nombre (propiedad: valor), entre llaves.	`{ reps: 5, porcentaje: 65 }`
Template string	Texto entre comillas invertidas donde ${...} se sustituye por el valor de una variable.	`` `Hola ${nombre}` ``
textContent	El texto de un elemento. Lo trata siempre como texto plano, nunca como HTML (es lo más seguro).	`titulo.textContent = "Hola"`
replaceChildren()	Quita todos los hijos de un elemento. Sin argumentos, lo deja vacío.	`tbody.replaceChildren();`
insertRow()	Método de las tablas: crea una fila nueva y la añade ya dentro de la tabla.	`const fila = tbody.insertRow();`
insertCell()	Método de las filas: crea una celda nueva y la añade ya dentro de la fila.	`fila.insertCell()`

4. Parte A: el mensaje de error del campo RM

Este bloque personaliza el aviso que muestra el navegador cuando el RM está fuera de rango. Para que funcione, el <input> del HTML debe tener los atributos **min** y **max**.

Código	¿Qué significa?
`// MENSAJE DE ERROR PERSONALIZADO`	Los comentarios empiezan con //. El navegador los ignora; sirven para que tú (o quien lea) entiendas el código.
`const rm = document.getElementById("rm");`	Busca el elemento con id="rm" (el campo del RM) y lo guarda en la variable rm. Desde ahora rm representa ese campo, y lo reutilizaremos más abajo.
`rm.addEventListener("input", () => {`	Dice: "cada vez que cambie lo escrito en este campo (evento input), ejecuta lo que hay entre las llaves". Se ejecuta con cada tecla pulsada.
`    rm.setCustomValidity("");`	Limpia el mensaje de error anterior. Si no lo hicieras, el campo seguiría marcado como inválido aunque corrigieras el número.
`    if (rm.validity.rangeUnderflow) {`	rm.validity es el informe de estado del campo. rangeUnderflow vale true cuando el número es menor que el min del HTML.
`        rm.setCustomValidity("Ese peso es muy bajo...");`	Pone un mensaje de error propio. Mientras exista, el navegador no deja enviar el formulario y lo muestra.
`    } else if (rm.validity.rangeOverflow) {`	Si no fue demasiado bajo, comprueba lo contrario: rangeOverflow es true cuando el número es mayor que el max.
`        rm.setCustomValidity("Ese peso es muy alto...");`	Mensaje propio para el caso de número demasiado alto.
`    }` `});`	Cierran el if y la función flecha. }); termina también la llamada a addEventListener.

Idea clave: setCustomValidity("") con texto vacío significa "el campo es válido". Con cualquier texto significa "el campo es inválido y este es el error".

Código completo de esta parte:

```
// MENSAJE DE ERROR PERSONALIZADO PARA EL INPUT DEL RM
const rm = document.getElementById("rm");

rm.addEventListener("input", () => {
    rm.setCustomValidity("");   // limpia el mensaje anterior

    if (rm.validity.rangeUnderflow) {
        rm.setCustomValidity("Ese peso es muy bajo. Sube un poco el RM.");
    } else if (rm.validity.rangeOverflow) {
        rm.setCustomValidity("Ese peso es muy alto. Revisa el RM.");
    }
});
```

5. Parte B: guardar los elementos de la página

Antes de trabajar, el código agarra los elementos que va a necesitar y los guarda en variables. Así no tiene que buscarlos cada vez.

Código	¿Qué significa?
`const form = document.getElementById("emomForm");`	Guarda el formulario completo (el elemento con id="emomForm").
`const resultado = document.getElementById("resultado");`	Guarda el contenedor que se mostrará con el resultado.
`const resultadoTexto = document.getElementById("resultadoTexto");`	Guarda el texto-título del resultado, por ejemplo "EMOM de 14 minutos: ...".
`const tbody = document.querySelector("#tablaEmom tbody");`	Busca con selector CSS el <tbody> que está dentro de #tablaEmom. Es el cuerpo de la tabla, donde se añadirán las filas.

Fíjate: cada id debe estar escrito exactamente igual en el HTML y en el JavaScript, incluidas las mayúsculas. Si no existe, el resultado es null y verás un error en la consola.

6. Parte C: la lista de fases (FASES)

FASES es un **array** (lista) con 7 **objetos**. Cada objeto describe un tramo de 2 minutos del EMOM. Se escribe en mayúsculas por convención, para indicar que es una constante que no cambia.

```
const FASES = [
    { minutos: "Min 1 - 2",   porcentaje: 65, reps: 5 },
    { minutos: "Min 3 - 4",   porcentaje: 70, reps: 4 },
    { minutos: "Min 5 - 6",   porcentaje: 75, reps: 3 },
    { minutos: "Min 7 - 8",   porcentaje: 80, reps: 2 },
    { minutos: "Min 9 - 10",  porcentaje: 85, reps: 1 },
    { minutos: "Min 11 - 12", porcentaje: 90, reps: 1 },
    { minutos: "Min 13 - 14", porcentaje: 95, reps: 1 }
];
```

Propiedad	Qué guarda	Tipo
`minutos`	Etiqueta del tramo que se muestra en la tabla.	Texto
`porcentaje`	Qué porcentaje de tu RM debes levantar en ese tramo.	Número
`reps`	Cuántas repeticiones hacer cada minuto.	Número

Para acceder a un dato: con un objeto **fase** escribes `fase.porcentaje` (punto + nombre de la propiedad) y obtienes, por ejemplo, 65.

7. Parte D: qué pasa al enviar el formulario

Este es el bloque más importante. Se ejecuta cuando alguien pulsa el botón de enviar. Lo dividimos en cuatro tablas.

7.1 Empezar y leer los datos

Código	¿Qué significa?
`form.addEventListener("submit", (e) => {`	Escucha el evento submit (enviar formulario). e es el objeto evento: contiene información de lo que ha ocurrido.
`    e.preventDefault();`	Los formularios, por defecto, recargan la página al enviarse. Esto lo cancela para que JavaScript controle todo sin recargar.
`    const ejercicio = document.getElementById("ejercicio").value;`	Lee el ejercicio elegido en el desplegable. .value es el valor de la opción seleccionada. Antes eran dos líneas; ahora basta con ésta.
`    const rmKg = Number(rm.value);`	Reutiliza la variable rm de la Parte A. .value siempre devuelve texto, aunque escribas un número; Number(...) lo convierte en número real para poder multiplicar.

Ojo: .value devuelve lo que pone en el atributo value de la opción. Si en tu HTML el value es distinto del texto que se ve (por ejemplo value="bs" y texto "Back squat"), y quieres mostrar el texto visible, usa `select.selectedOptions[0].textContent`, donde select es el desplegable.

7.2 Preparar el título y vaciar la tabla

Código	¿Qué significa?
`    resultadoTexto.textContent = `EMOM de 14 minutos: ${ejercicio} (RM: ${rmKg} kg)`;`	Cambia el texto del título. Las comillas invertidas forman un template string: ${ejercicio} y ${rmKg} se sustituyen por sus valores. Ejemplo: "EMOM de 14 minutos: Back squat (RM: 100 kg)".
`    tbody.replaceChildren();`	Vacía el cuerpo de la tabla. Sin esto, cada vez que envías el formulario se añadirían filas debajo de las anteriores. Hace lo mismo que tbody.innerHTML = "" pero dice claramente lo que hace.

7.3 Recorrer las fases y construir la tabla

Código	¿Qué significa?
`    FASES.forEach((fase) => {`	forEach recorre la lista. Ejecuta el bloque una vez por cada fase. En cada vuelta, fase es un objeto distinto (la 1ª, la 2ª...).

`        const peso = Math.round(`
`            (rmKg * fase.porcentaje`
`            / 100) / 2.5`
`        ) * 2.5;`
	Calcula el peso: rmKg * fase.porcentaje / 100 es el porcentaje del RM. Luego se divide entre 2,5, se redondea con Math.round y se multiplica por 2,5. Resultado: el múltiplo de 2,5 kg más cercano, que es lo que se puede cargar con discos.

Código	¿Qué significa?
`const fila = tbody.insertRow();`	Crea una fila nueva y ya la mete al final del tbody. Ya no hace falta createElement ni appendChild.
`fila.insertCell().textContent = fase.minutos;`	Crea una celda dentro de la fila y le escribe el tramo de minutos. Al usar textContent, el valor se trata como texto, nunca como HTML.
`fila.insertCell().textContent = `${fase.reps} ${fase.reps === 1 ? "Rep" : "Reps"}`;`	Segunda celda: las repeticiones. La parte fase.reps === 1 ? "Rep" : "Reps" es un operador ternario: "si reps es igual a 1, escribe Rep; si no, escribe Reps". Así sale "1 Rep" pero "5 Reps".
`fila.insertCell().textContent = `${peso} kg`;`	Tercera celda: el peso calculado.
`});`	Cierra el forEach. Se repite hasta acabar las 7 fases.

7.4 Mostrar todo

Código	¿Qué significa?
`resultado.hidden = false;`	El atributo hidden oculta un elemento. Al ponerlo en false el contenedor del resultado se hace visible.
`document.getElementById("tablaEmom").hidden = false;`	Hace visible la tabla (en el HTML empieza oculta con hidden).
`document.getElementById("linkhidden").hidden = false;`	Hace visible el enlace a la imagen del EMOM.
`});`	Cierra la función del evento submit y la llamada a addEventListener.

Código completo de esta parte:

```
form.addEventListener("submit", (e) => {
    e.preventDefault();

    const ejercicio = document.getElementById("ejercicio").value;
    const rmKg = Number(rm.value);

    resultadoTexto.textContent = `EMOM de 14 minutos: ${ejercicio} (RM: ${rmKg} kg)`;

    tbody.replaceChildren();

    FASES.forEach((fase) => {
        const peso = Math.round((rmKg * fase.porcentaje / 100) / 2.5) * 2.5;

        const fila = tbody.insertRow();
        fila.insertCell().textContent = fase.minutos;
        fila.insertCell().textContent = `${fase.reps} ${fase.reps === 1 ? "Rep" : "Reps"}`;
        fila.insertCell().textContent = `${peso} kg`;
    });

    resultado.hidden = false;
    document.getElementById("tablaEmom").hidden = false;
    document.getElementById("linkhidden").hidden = false;
});
```

8. Ejemplo con números reales

Supón que escribes RM = 87 kg. Vamos a calcular el peso de la primera fase (65 %):

Paso	Operación	Resultado
1	87 × 65 / 100	56,55
2	56,55 / 2,5	22,62
3	Math.round(22,62)	23
4	23 × 2,5	57,5 kg

Con RM = 100 kg todo da múltiplos exactos de 2,5, así que la tabla sale así:

Minutos	Reps	Peso
1 - 2	5 Reps	65 kg
3 - 4	4 Reps	70 kg
5 - 6	3 Reps	75 kg
7 - 8	2 Reps	80 kg
9 - 10	1 Rep	85 kg
11 - 12	1 Rep	90 kg
13 - 14	1 Rep	95 kg

9. El HTML que necesita este código

El JavaScript solo funciona si el HTML tiene estos elementos con estos id. Es un esqueleto mínimo de referencia (la tabla tiene tres columnas, igual que las celdas que crea el código):

```
<form id="emomForm">
    <select id="ejercicio">
        <option value="Back squat">Back squat</option>
        <option value="Deadlift">Deadlift</option>
        <option value="Push press">Push press</option>
        <option value="Front squat">Front squat</option>
    </select>
    <input id="rm" type="number" min="45" max="400" required>
    <button type="submit">Generar</button>
</form>

<div id="resultado" hidden>
    <p id="resultadoTexto"></p>
    <table id="tablaEmom" hidden>
        <thead>
            <tr><th>Minutos</th><th>Reps</th><th>Peso</th></tr>
        </thead>
        <tbody></tbody>
    </table>
    <a id="linkhidden" href="emom.png" hidden>Ver imagen</a>
</div>

<script src="js/script.js" defer></script>
```

Nota: sin min y max en el input, los avisos rangeUnderflow y rangeOverflow nunca saltan. Y el atributo **defer** en el script hace que se ejecute cuando el HTML ya está cargado, así los elementos no salen como null.

10. Qué ha cambiado respecto a la versión anterior

Antes	Ahora	Por qué
`tbody.innerHTML = "";`	`tbody.replaceChildren();`	Dice claramente lo que hace y no pasa por el parser de HTML.
`document.createElement("tr") + appendChild(fila)`	`tbody.insertRow()`	Una sola línea; la fila ya queda dentro de la tabla.
`fila.innerHTML = `<td>...</td>...``	`fila.insertCell().textContent = ...`	textContent es texto plano: no se puede colar HTML por error, y es un hábito más seguro para el futuro.
`select.options[select.selectedIndex].text`	`document.getElementById("ejercicio").value`	Menos pasos. Si necesitas el texto visible: selectedOptions[0].textContent.
`Number(document.getElementById("rm").value)`	`Number(rm.value)`	Reutiliza la variable rm que ya existe. Menos código repetido.
`Mensajes de error con insultos`	`Mensajes neutros`	Funciona igual y se puede enseñar a cualquiera.

Importante: innerHTML no está obsoleto ni prohibido; es estándar y funciona. Simplemente, para vaciar una tabla y rellenarla con datos, las opciones de arriba son más claras y más seguras.

11. Errores típicos de principiante

Error	Síntoma	Solución
Escribir mal un id	La consola muestra "Cannot read properties of null".	Comparar el id del HTML con el del JS, letra por letra.
Olvidar Number()	Los cálculos fallan o se concatenan como texto.	Convertir siempre .value antes de calcular.
Quitar e.preventDefault()	La página se recarga y la tabla desaparece.	Mantenerlo como primera línea del submit.
Olvidar tbody.replaceChildren()	Las filas se acumulan en cada envío.	Vaciar la tabla antes de rellenarla.
No limpiar setCustomValidity("")	El campo queda inválido para siempre.	Limpiar al inicio del evento input.
Cargar el script antes del HTML	Los elementos salen como null.	Poner <script> al final del <body> o usar defer.
Poner más celdas que columnas	La tabla se ve descuadrada.	Que el número de insertCell() coincida con los <th>.

12. Mini test de repaso

Intenta responder sin mirar. Las soluciones están justo debajo.

1. ¿Para qué sirve e.preventDefault()?

2. ¿Por qué hay que usar Number() con el valor del input?

3. ¿Qué hace Math.round(x / 2.5) * 2.5?

4. ¿Por qué se usa textContent en lugar de innerHTML para rellenar las celdas?

5. ¿Qué hace tbody.insertRow()?

6. ¿Qué hace tbody.replaceChildren() y por qué se llama antes del forEach?

7. ¿Qué pasa si hidden se pone en false?

Soluciones

N.º	Respuesta
1	Evita que el navegador recargue la página al enviar el formulario, para que JavaScript controle el proceso.
2	Porque .value devuelve texto. Con Number() se convierte en número y se puede multiplicar y redondear.
3	Redondea x al múltiplo de 2,5 más cercano.
4	Porque textContent trata el valor como texto plano. innerHTML interpreta etiquetas HTML, lo que es innecesario aquí y menos seguro.
5	Crea una fila nueva y la añade al final del tbody, lista para meterle celdas con insertCell().
6	Vacía el cuerpo de la tabla. Se llama antes del forEach para que las filas de envíos anteriores no se acumulen.
7	El elemento deja de estar oculto y se muestra.